

Spring 26 CSCI 240 -- PA 8

Binary Search Trees (BST) & AVL Trees

Feel free to discuss and help each other out but does not imply that you can give away your code or your answers! Make sure to read all instructions before attempting this lab.

You can work with a lab partner and each one must submit the same PDF file (include both names in the submission file). Each person must include a brief statement about your contribution to this assignment.

You must use an appropriate provided template from Canvas and output "Author: Your Name(s)" for all your programs. If you are modifying an existing program, use "Modified by: Your Name(s)".

Exercise 1: You must use **TreeMap** class in C++ book or **TreeMap** class in Java book and set up a test driver to perform some operations such as insert, erase, and find. Perform the operations in question 1 below (steps 1 to 7) and then search for 15, 30, and 8. Print the BST as the final step. Assume that key is an integer and value is a string such as a name (come up with your own names).

Exercise 2: You will implement a simple population database for California counties using a simple search tree from exercise 1 to store the database records. Define and implement **SimplePopMap** class that supports standard map operations using county code as a key for each record (no duplicate keys). Your **SimplePopMap** class uses binary search tree to store population records. Download the data file *popSmall.txt*, containing a list of a few population records (first line is just the number of records in the file) – county code, population in million, and county with state abbreviation (3 fields separated by commas with first field is the key and the remaining fields are value, and you can treat those remaining fields as one string). Build the search tree from the records of the input data file by inserting one record at a time into the tree. Run the following test cases after the tree is constructed:

1. List all records
2. Search for 6037
3. Search for 6000
4. Insert 6066, "1, New County, CA"
5. Insert 6065, "2000, Riverside, CA"
6. Delete 6999
7. Delete 6075
8. Delete 6055
9. List all records

```

Class SimplePopMap
    // private data
    // must use TreeMap (C++) or TreeMap (Java)

    // public operations
    // constructor accepts file name and construct search tree
    // print the height of the tree at the end
    SimplePopMap(string filename)

    // print appropriate message and data if found/not found
    void find(int code)

    // print appropriate message and insert node if not found
    // replace data if found
    void insert(int code, string popCounty)

    // print appropriate message and erase node if found
    void erase(int code)

    // print one record per line using an in-order traversal;
    // also print the height of the tree
    void print()

```

Exercise 3: You must use **AVLTreeMap** class in C++ book or **AVLTreeMap** class in Java book (modify if needed) and set up a test driver to perform some operations such as insert, erase, and find. Perform the operations in question 1 below (steps 1 to 7) and then search for 15, 30, and 8. Print the AVL tree as the final step (include key, value, and height of each node). Assume that key is an integer and value is a string such as a name (come up with your own names).

Exercise 4: You will implement a better population database for California counties using an AVL tree from exercise 1 to store the database records. Define and implement **BetterPopMap** class that supports standard map operations using county code as a key for each record (no duplicate keys). Your **BetterPopMap** class uses an AVL tree to store population records. Download the data file *popSmall.txt*, containing a list of a few population records (first line is just the number of records in the file) – county code, population in million, and county with state abbreviation (3 fields separated by commas with first field is the key and the remaining fields are value, and you can treat those remaining fields as one string). Build the AVL tree from the records of the input data file by inserting one record at a time into the tree. Run the following test cases after the tree is constructed:

1. List all records
2. Search for 6037
3. Search for 6000
4. Insert 6066, "1, New County, CA"
5. Insert 6065, "2000, Riverside, CA"
6. Delete 6999
7. Delete 6075
8. Delete 6055

9. List all records

```
Class BetterPopMap
// private data
// must use AVLTreeMap (C++) or AVLTreeMap (Java)

// public operations
// constructor accepts file name and construct AVL tree
// print the height of the tree at the end
BetterPopMap(string filename)

// print appropriate message and data if found/not found
void find(int code)

// print appropriate message and insert node if not found
// replace data if found
void insert(int code, string popCounty)

// print appropriate message and erase node if found
void erase(int code)

// print one record per line using an in-order traversal;
// need to print height of each node to the right of each node
void print()
```

Question 1: Provide the final BST after the following operations (showing keys only). You might want to draw multiple trees along the way to show your work.

1. Insert 10
2. Insert 20
3. Insert 4
4. Insert 8
5. Insert 15
6. Erase 8
7. Erase 10

Question 2: Provide the AVL tree after the following operations (showing keys only). You might want to draw multiple trees along the way to show your work.

Insert 10, Insert 20, Insert 30, Insert 15, Insert 12, Insert 20, Erase 30

You can earn EC for only one of the options below:

Extra Credit Option 1 (4 points): Add code to exercises 2 & 4 to print the average number of operations (a floating-point value for number of nodes examined) to perform each insert operation of all data from the file. You will need to modify some files in the

book. Try both the small file, popSmall.txt, and the larger file, popLarge.txt. You can submit one version here to include exercise 2 and one version here to include exercise 4.

Extra Credit Option 2 (2 points): Add public operation drawTree() to BetterPopMap class in exercise 4 to draw the tree (key only) for the small data file. You can submit one version here to include exercise 4. See sample drawing below where node 1234 is the root with left child 1000 and node 1000 has no children. Node 2000 is the right child of the root with 2 children, 1500 and 2000, and those children have no children.

```
1234
  1000
   2000
    1500
    2000
```

Fill out and turn in the PA submission file for this assignment (save as PDF format).